AutoVet Software Testing Activities

Unit, Integration, System, Security and Performance Testing

Test runs executed on November 11, 2026

API testing tool	Postman collections, run with Newman 6.2.1 (Postman's command-line collection runner)
Backend unit tests	PHPUnit 11 via `php artisan test` (Laravel 12, PHP 8.2.35)
Frontend tests	Vitest 3.2.4 + React Testing Library 16.3 (React 19 admin app)
Load testing	Node.js load script (`docs/postman/testing/load-test.js`)
Database	MySQL 8.4: a separate QA copy of the clinic database (584 appointments, 40 inventory items, 1,369 usage records)
AI module	Python 3.13, scikit-learn 1.8 LinearRegression (`backend/ai/forecast.py`)
Server	Laravel API with 4 PHP workers and opcache enabled, the same worker count and opcache settings as the production server

1. Unit Testing

Individual components and functions were tested in isolation to confirm that each one produces the correct output for a given input.

Testing Activity: Backend Business Logic (PHPUnit)

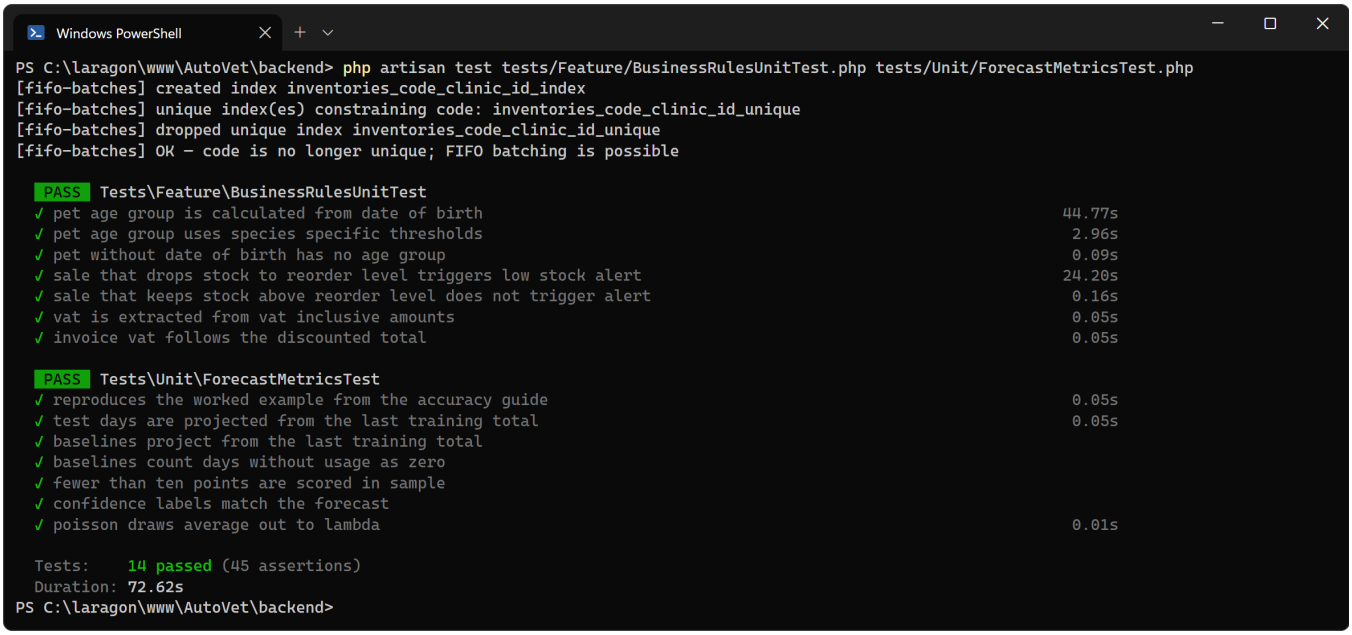


Figure 1. PHPUnit results for the Laravel backend

The figure presents the backend unit test results using PHPUnit. Tests were written for the system's critical business logic. **Pet age calculation:** the age group (Baby, Young, Adult or Senior) is computed from the date of birth using species-specific thresholds, so an 8-year-old dog is Senior while an 8-year-old cat is still Adult. **Stock level triggers:** finalizing a sale that brings stock down to the reorder level raises a low-stock alert, and a sale that keeps stock above it does not. **Tax calculation:** since clinic prices already include 12% VAT, the VAT portion is extracted correctly (₱120 inside ₱1,120) and follows the discounted total. The forecast accuracy formulas (R^2, RMSE, MAE and confidence labels) were also verified. All 14 tests passed with 45 assertions and no failures.

Testing Activity: Frontend Components (Vitest + React Testing Library)

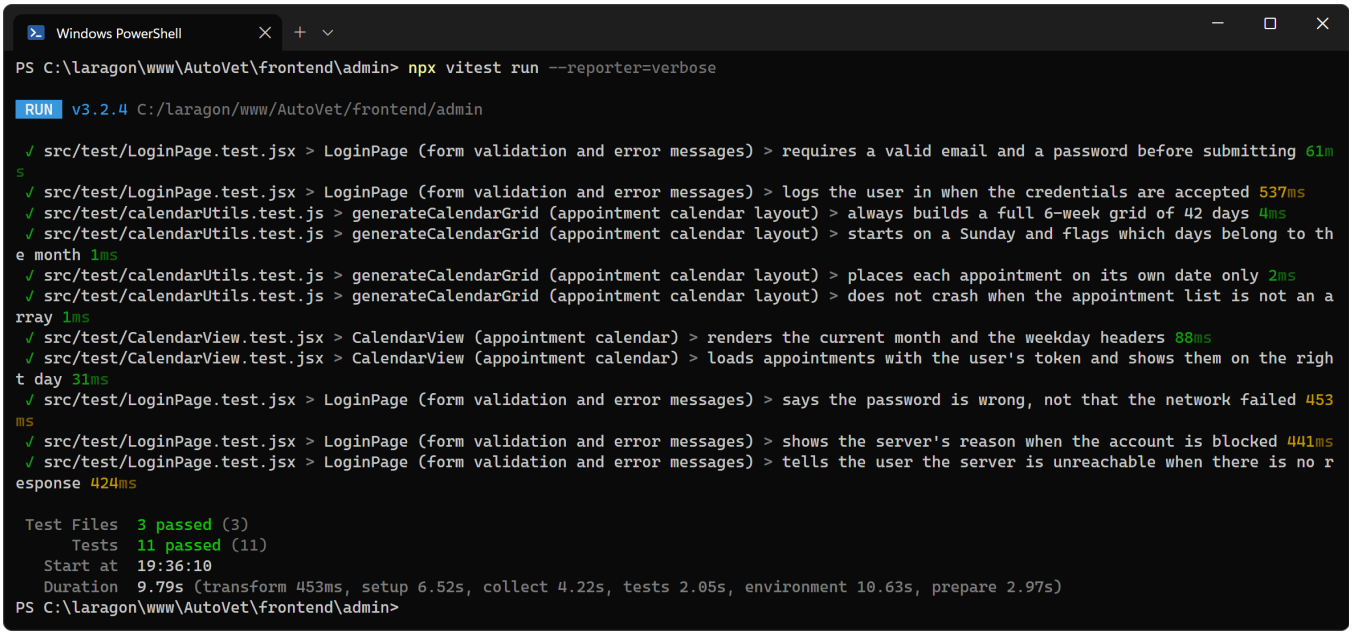


Figure 2. Component tests for the React admin app

The figure presents the frontend component tests using Vitest and React Testing Library. The **appointment calendar** was tested to confirm that it renders the current month and weekday headers, loads appointments using the logged-in user's token, places each appointment on its correct date, and always builds a full six-week grid. The **login form** was tested for input validation (a required email-type field and a required password) and for accurate feedback: a wrong password shows "Incorrect email or password.", a blocked account shows the server's reason, and an unreachable server is reported as such. All 11 tests in 3 test files passed.

2. Integration Testing

Integration testing focused on the interaction between the different modules of the AutoVet system.

Testing Activity: AI Forecasting Module Integration (Postman)

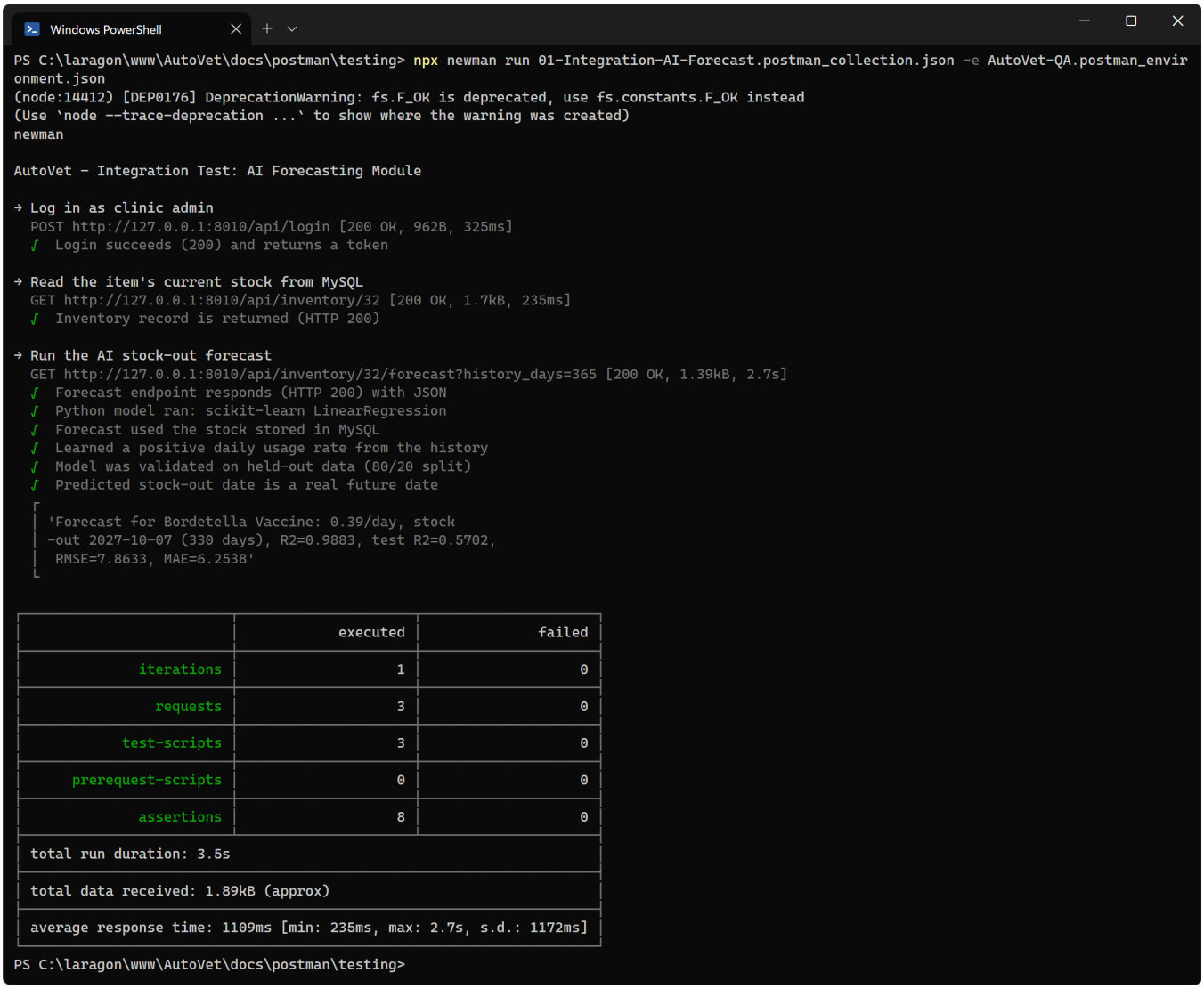


Figure 3. Postman collection run: Laravel API → MySQL → Python forecast → JSON

The figure presents the integration test of the AI module, run as a Postman collection. The test first reads the inventory item's stock from the MySQL database. It then calls the forecast endpoint, where Laravel exports the item's usage history from MySQL, runs the Python forecasting script (scikit-learn LinearRegression), and returns the prediction as JSON. Postman confirmed that the response came from the Python model, that it used the exact stock stored in MySQL, and that it was validated on held-out data (80/20 split). For Bordetella Vaccine the model learned a usage rate of 0.39 units per day and predicted a stock-out in 330 days (October 7, 2027), with a trend fit of $R^2 = 0.99$. All 8 assertions passed.

3. System Testing

End-to-end testing of the whole application, simulating real-world clinic operations.

Testing Activity: Scenario-Based "Patient Journey" (Postman), Part 1

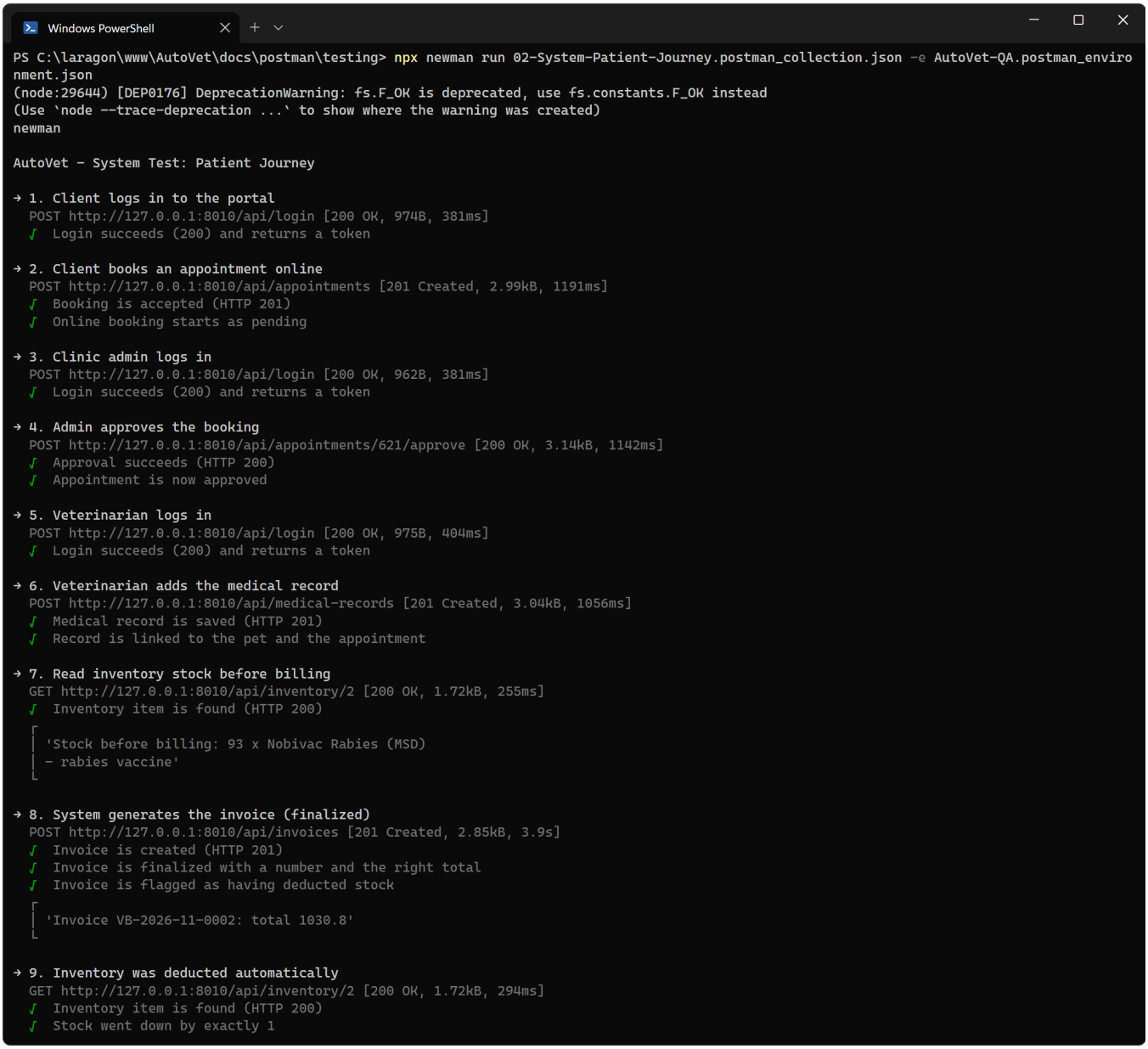


Figure 4. Patient journey, steps 1 to 9

Testing Activity: Scenario-Based "Patient Journey" (Postman), Part 2

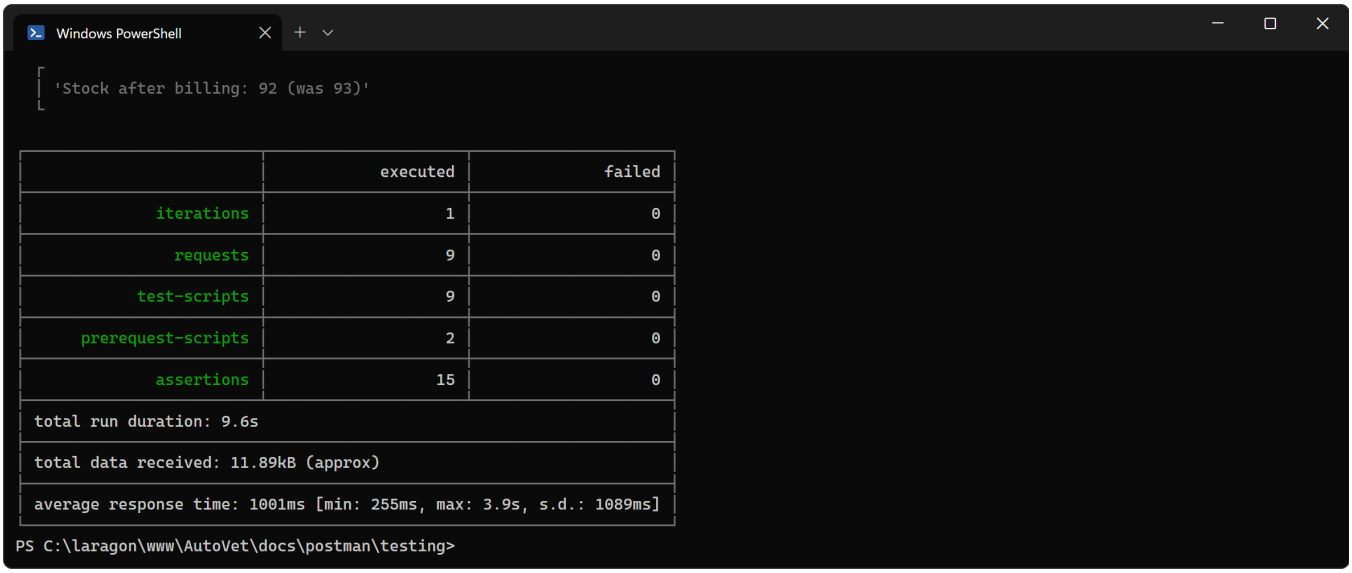


Figure 5. Patient journey, stock check result and run summary

---

The figures present the full patient journey, run as one Postman collection with each step performed by the correct role. **The client books online:** a pet owner logs in to the portal and books an appointment, which starts as pending. **The admin approves:** the clinic admin approves the booking. **The veterinarian adds a medical record:** a record linked to the pet and the appointment is saved. **The system generates the invoice:** a finalized invoice (VB-2026-11-0002, ₱1,030.80) is created for the consultation and an anti-rabies vaccine. **Inventory deducts automatically:** the vaccine's stock went from 93 to 92 without any manual adjustment. All 9 requests and 15 assertions passed.

4. Security & Performance Testing

Testing Activity: Role-Based Access Control (Postman), Part 1

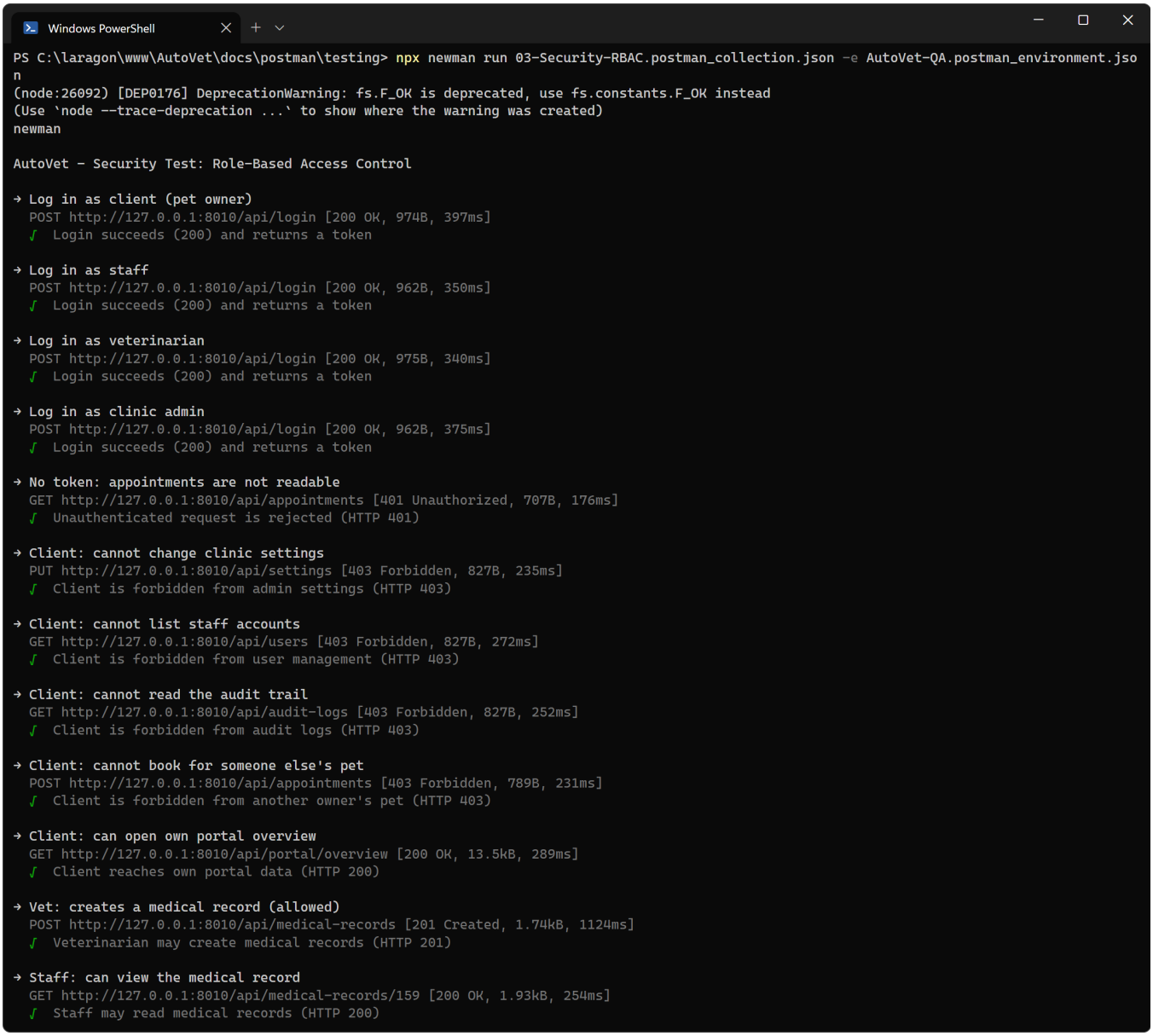


Figure 6. Logins, then access checks for unauthenticated users, clients, veterinarians and staff

Testing Activity: Role-Based Access Control (Postman), Part 2

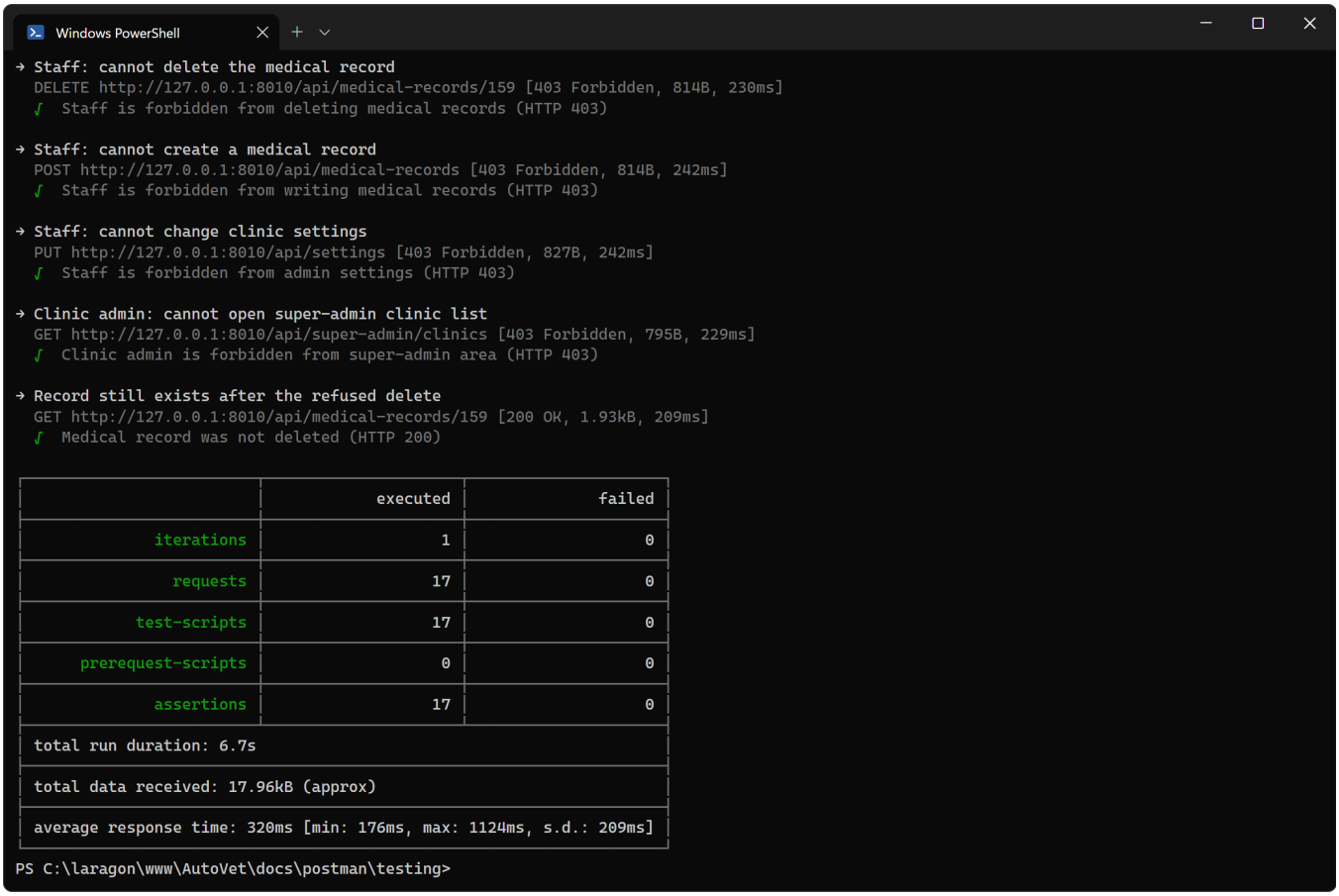


Figure 7. Remaining staff and clinic admin checks, and the run summary

---

The figures present the role-based access control test, run as a Postman collection that logs in as a client, staff member, veterinarian and clinic admin. A **client** was refused (HTTP 403) when trying to change clinic settings, view staff accounts, read the audit trail, or book for another owner's pet, but could still open their own portal. **Staff** could view a medical record but were refused when trying to delete it, create one, or change clinic settings. A **clinic admin** was refused access to the super-admin area, and requests with no login were rejected (HTTP 401). A final check confirmed the medical record still existed after the refused delete. All 17 requests and 17 assertions passed.

Testing Activity: Simultaneous User Access and System Performance

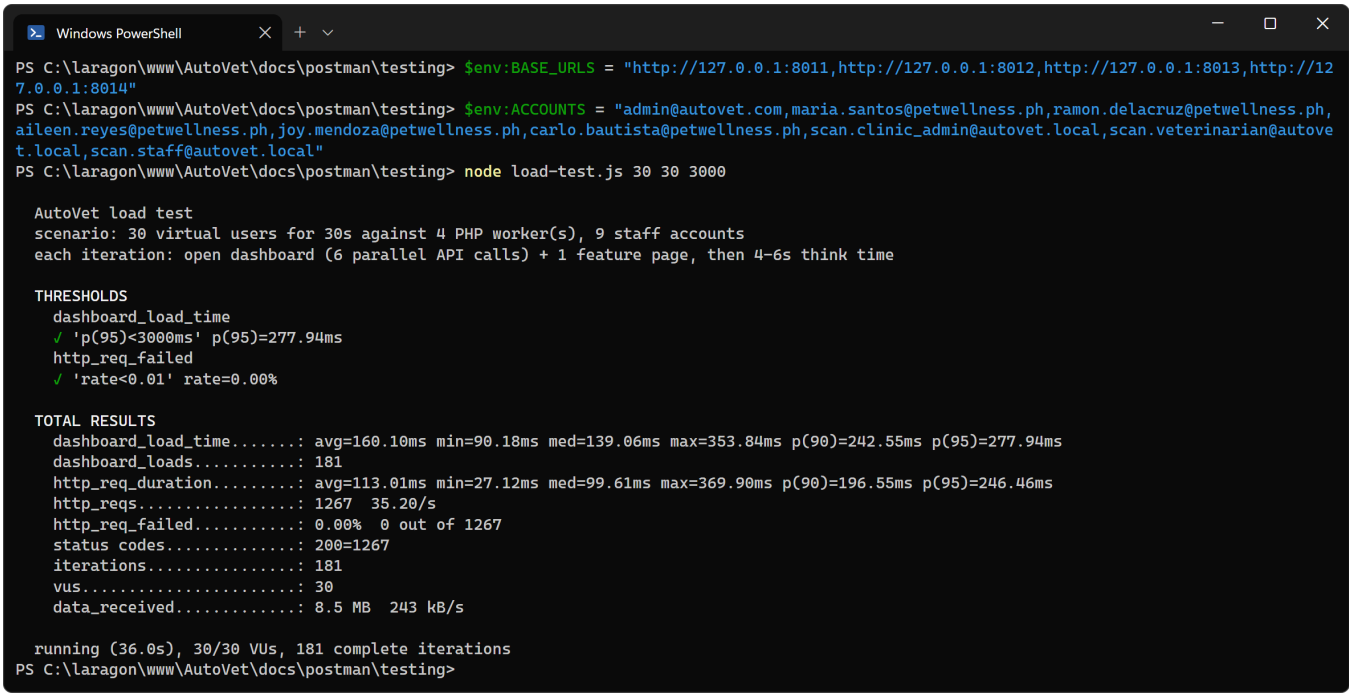


Figure 8. Load test with 30 simultaneous users

The figure presents the load test, which simulated 30 clinic staff using the system at the same time for 30 seconds. Each virtual user logged in with a real staff account and repeatedly opened the dashboard (its 6 API calls sent in parallel, as the browser does), opened another feature page (appointments, inventory, clients, pets or invoices), then paused 4–6 seconds like a person reading the screen. The system processed 1,267 requests with a 100% success rate and a 0.00% error rate. The dashboard loaded in 160 milliseconds on average, and 95% of dashboard loads finished within 278 milliseconds, well within the target of under 3 seconds.